| | |
|---|---|
| **Número del equipo:** 8 | |
| **Nombres**:<br>Esther Garza Cano<br>Luis Ángel David Cavazos Reyes<br>Emilio Garza Arizpe | **Matrícula**:<br>2143077<br>2142838<br>2143702 |
| **Nombre del curso:** Componentes de Lógica de Negocio | **Nombre del profesor**: Miguel Ángel Gómez Marroquín |
| **Actividad**: Elegir patrones AulaPlus | |
| **Fecha**: 02/10/2026 | |
| **Bibliografia (Formato APA)** | |

4.3 Registros de ADR

ADR-001: Selección del Patrón Domain Model sobre Transaction Script

| Atributo | Detalle |
|---|---|
| **Estatus** | **Aceptado** |
| Contexto | El sistema *Aula Plus* debe gestionar reglas de negocio complejas como la validación de cancelación ($\geq 48$h), la verificación de duración de clase ($\geq 70\%$) y la distribución inmutable de tarifa ($200 / $40 / $160). Un enfoque centrado en base de datos provocaría dispersión de lógica. |
| Decisión | Adoptar el patrón de arquitectura de lógica de negocio **Domain Model (Modelo de Dominio)** complementado con Domain-Driven Design (DDD) |
| Consecuencias Positivas | • Alta encapsulación de reglas e invariantes dentro de entidades y objetos de valor.<br>• Facilidad de pruebas unitarias al no depender de la base de datos.<br>• Alineación directa con el Lenguaje Ubicuo del negocio. |
| Consecuencias Negativas | • Mayor curva de aprendizaje y complejidad inicial en el mapeo de persistencia (ORM). |

ADR-002: Descoplamiento de Lectura y Escritura mediante CQRS Básico

| Atributo | Detalle |
|---|---|
| **Estatus** | **Aceptado** |
| Contexto | Las operaciones de consulta (ej. historial de tutorías o catálogo de tutores) requieren alta velocidad y DTOs planos, mientras que las operaciones de modificación demandan cargar agregados pesados para validar invariantes. |
| Decisión | Implementar **CQRS Básico (Command Query Responsibility Segregation)**, separando los comandos (*Commands*) que pasan por la capa de aplicación y el modelo de dominio, de las consultas (*Queries*) que leen directamente proyecciones optimizadas DTO. |
| Consecuencias Positivas | • Optimización de rendimiento en consultas sin sobrecargar el modelo de dominio.<br><br>• Código más limpio y alineado al principio SOLID de Responsabilidad Única (SRP). |
| Consecuencias Negativas | • Duplicidad en la definición de estructuras de datos (DTOs de lectura vs. Entidades de escritura). |

ADR-003: Gestión Transaccional mediante el Patrón Unit of Work

| Atributo | Detalle |
|---|---|
| **Estatus** | **Aceptado** |
| Contexto | Casos de uso como notificarAusencia o procesarLiquidacion afectan a múltiples agregados (Tutor y Sesion) en una misma operación, requiriendo garantías ACID. |
| Decisión | Utilizar el patrón **Unit of Work (Unidad de Trabajo)** en la Capa de Servicios de Aplicación para rastrear cambios en memoria y realizar commits/rollbacks atómicos. |
| Consecuencias Positivas | • Garantiza consistencia transaccional sin acoplar la capa de aplicación a una tecnología SQL/NoSQL específica.<br><br>• Minimiza el número de llamadas a la base de datos agrupando escrituras. |
| Consecuencias Negativas | • Requiere implementar un mecanismo explícito de rastreo de objetos (*Change Tracking*) en memoria. |